

CSE 554: Systems for Machine Learning – Assignment 4

In this assignment, you will implement the continuous batching and chunked prefill to enhance the scheduler, as well as extend the pipeline to multiple GPUs. Your submission will be in the form of a .zip file upload on Canvas, and must include a **short report with answers to the following questions (.pdf)** and the **code**.

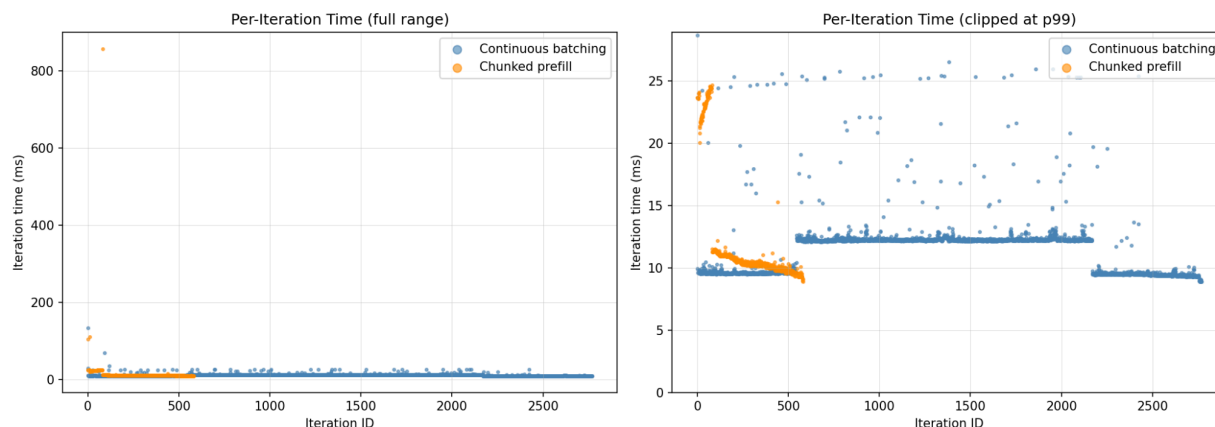
Section 1: Continuous Batching and Chunked Prefill

Q1. 10 + 10 points

- Implement the continuous batching in files `Section1/continous_engine.py` and `Section1/continuous_scheduler.py`
- With batch size 10, input length following a uniform distribution between [1,10], output length following a uniform distribution between [1, 128], profile the end-to-end execution time of naive scheduling and continuous batching serving 100 requests. What is the speedup over naive scheduling?
Naive scheduling completes 100 requests in 14.58 seconds, and continuous batching finishes them in 9.00 seconds. The speedup is 1.62x.

Q2. 10 + 10 + 10 points

- Implement chunked prefill in `Section1/chunked_engine.py` and `Section1/chunked_scheduler.py`. Control the maximum token batch size to be 512 in each iteration.
- With the input length following a lognormal distribution with $\mu = 6$, $\sigma = 0.7$, and the output length following a uniform distribution between [1, 512], profile the end-to-end execution time of the chunked prefill solution. How does it compare with continuous batching?
Chunked prefill finishes 100 requests in 7.62 seconds, and continuous batching finishes in 28.62 seconds when using a batch size of 10. Chunked prefill is 3.75x faster.
- Compare the iteration time of continuous batching and chunked prefill by plotting them as scatter plots, with iteration id as the x-axis. What is the difference?



Both scatter plots are drawn using the same data. The right one shows the bottom 99% of the data points (since in the left one these points are clustered together).

Observe that the general trend for chunked prefill is some slightly longer iterations (20-25ms) followed by slightly shorter iterations (around 10ms). The early iterations are longer due to chunked prompt ingestion, while the later iterations are mostly decoding.

Continuous batching runs many more iterations (since the batch size at any time is restricted to 10). Its iteration times keep fluctuating around 10-25ms. The iteration time rises when new requests join and prefill work needs to be done, and the time drops when the engine performs only decoding.

For both chunked prefill and continuous batching, there are a few spikes, likely caused by large prefills.

In summary, chunked prefill runs fewer iterations and exhibits two phases: slightly longer iterations with chunked prefills and short iterations that are decode-only. Continuous batching runs many more iterations and shows more variation in iteration time throughout the run.

Q3. 10 + 10 points

- Profile vLLM performance with llama-3.2-1B, with input length 512, output length 512, request number 100. You can use the benchmark command here <https://docs.vllm.ai/en/stable/cli/bench/throughput/> vLLM takes 8.29 seconds.
- Profile your own chunked prefill performance with input length 512, output length 512, and 100 requests. How does that compare with vLLM? Which part (attention, FFN, other small kernels, overheads, etc) of vLLM is faster than your own implementation? Our chunked prefill takes 10.35 seconds. vLLM is 1.25x faster. Our chunked prefill implementation spends 4.15 seconds (40% of total time) on attention, 3.46 seconds (33%) on FFN, and 1.40 seconds (14%) on other GPU computation, with the remaining 0.55 seconds (5%) being overheads. One reason that vLLM's faster is because it uses CUDA graphs. Since inside one CUDA graph, there can be both FFN and attention computation, it is hard to measure the exact breakdown. But overall, attention and FFN take about 7.30 seconds together (88%), with the remainder being other small kernels (0.80 seconds, 10%) and overheads (2%).

In conclusion, vLLM is faster than our chunked prefill in every category shown here, thanks to its use of CUDA graphs, better optimized kernels, lower system overhead, etc.

Section 2: Parallelism

GPU assignment for this question: You need two GPUs for this question. If you were assigned GPU index i (0-indexed) in previous assignments, use the following GPU pair:

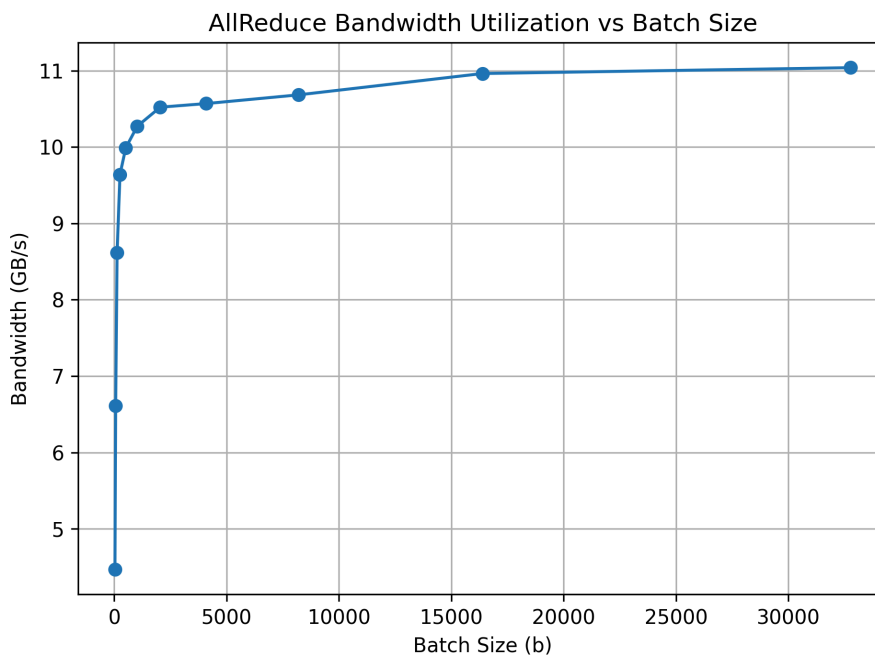
- GPUs: $(i // 2) * 2$ and $(i // 2) * 2 + 1$

Examples:

- If $i = 0$, use GPUs 0 and 1
- If $i = 5$, use GPUs 4 and 5

Q4. 10 + 10 + 10 + 10 points

- Profile the network bandwidth utilization of PyTorch's all-reduce collective communication by running the script in [Section2/profile_allreduce.py](#). What trend do you observe from the bandwidth utilization figure generated? How does the achieved network bandwidth utilization compare to the theoretical network bandwidth?
 - To calculate theoretical network bandwidth, use `nvidia-smi topo -m` to output the network topology of the cluster



For batch sizes ≥ 2048 (≈ 8 MB all-reduce tensor size), the bandwidth levels off at around 11 GB/s, indicating network bandwidth saturation.

Checking `nvidia-smi topo -m`, the connection between neighboring GPUs (in our case, 4 and 5) is PIX, which means "Connection traversing at most a single PCIe bridge". The

PCIe generation is 3 (peak bandwidth 1GB/s) and there are 16 ports. The theoretical bandwidth is $1\text{GB/s} * 16 = 16\text{ GB/s}$.

The all-reduce saturated bandwidth 11GB/s is less than the theoretical peak bandwidth. During all-reduce operation, the Reduce op also takes time so the calculated bandwidth is less than optimal.

- You are given the transformer implementation for Lama-3.2-1B shown in class in `Section2/transformer-w3l1.py`. Now we want to extend this implementation to support running Lama-3.2-1B using tensor parallelism of degree 2. Implement the tensor-parallel version of the attention block in `Section2/transformer-tp2.py`. Note that Q/K/V projection weights are split by head, and the output projection weight is split by row (refer to [Lecture 14](#), Slide 32).
- Implement the tensor-parallel version of the feed-forward network in `Section2/transformer-tp2.py`. Note that the up and gate weights are split by column, and the down projection weight is split by row (refer to [Lecture 14](#), Slide 31). You are expected to see the same output as running the original single-GPU implementation in `Section2/transformer-w3l1.py`.
 - Run `torchrun --nproc-per-node=2 Section2/transformer-tp2-answer.py` to test your implementation. This will set up ranks and world size for 2 GPUs automatically

The two models output the same contents:

output string: The University of Michigan is a public research university in Ann Arbor, Michigan. It is the oldest institution of higher education in the state of Michigan and the largest university in the state. The university is a member of the Association of American Universities and is ranked among the top 10 public universities in the United States. The university is known for its research in the fields of engineering, medicine, and social sciences. It is also home to the University of Michigan Medical School, one of the oldest medical schools in the United States. The university

- Profile the generation time for the tensor-parallel version of the Lama-3.2-1B implementation and compare it with the generation time for the original implementation. Is the generation time faster for the tensor-parallel version? If so, what are the reasons that lead to the speed-up? If not, what are the reasons causing the overheads?

The average time of running the single GPU iteration is: 2.28 s

The average time of running the tp-2 iteration is: 2.67 s

The generation time of the single GPU version is shorter than the tensor-parallel version.

The reasons causing the overheads include

1. Decreasing the problem sizes of matrix multiplications will lead to smaller GFLOPs, and
2. There is extra overhead introduced by the collective communications, like the all-reduce latency, the synchronization time.